

CS 550 Operating Systems

Spring 2026

Paging: introduction

Address translation recap

- Dynamic relocation (base and bound)
 - Internal fragmentation
- Segmentation
 - External fragmentation
 - Complex free space management
- Paging
 - How can paging solve the problem of segmentation?

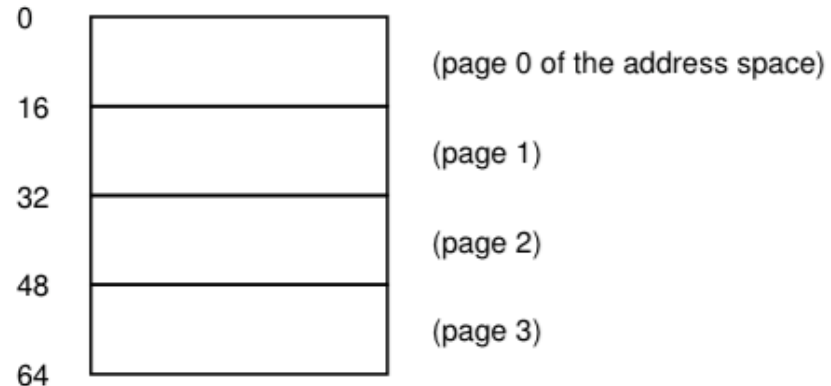
Paging: the idea

- Segmentation
 - split up the address space into three variable-sized logical segments
- Paging
 - split up the address space into fixed-sized units, which are called **pages**.

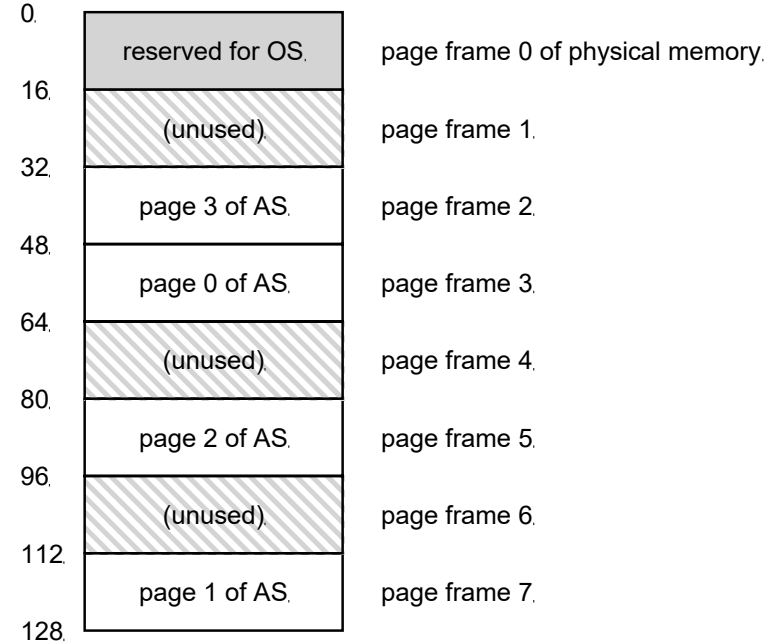
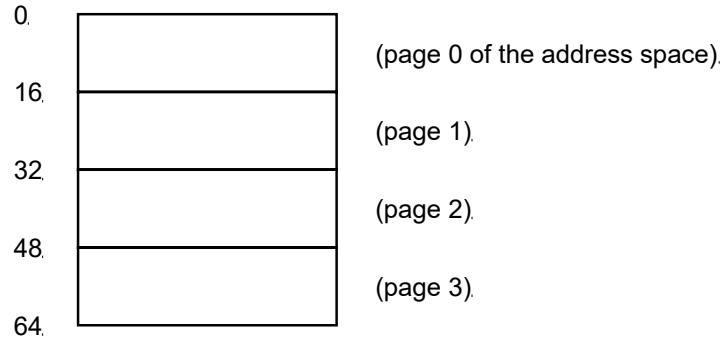
Example: a tiny address space

- 64 bytes in total
- 16 byte pages

(Reality: 32 bit address and
thus 4-GB of a address space)

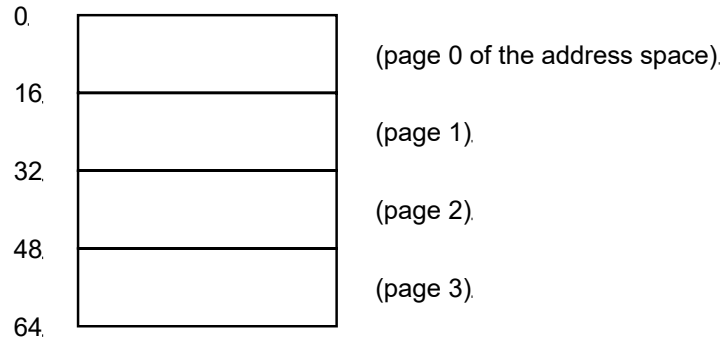


- With paging, physical memory is also split into some number of pages, called **page frames**.

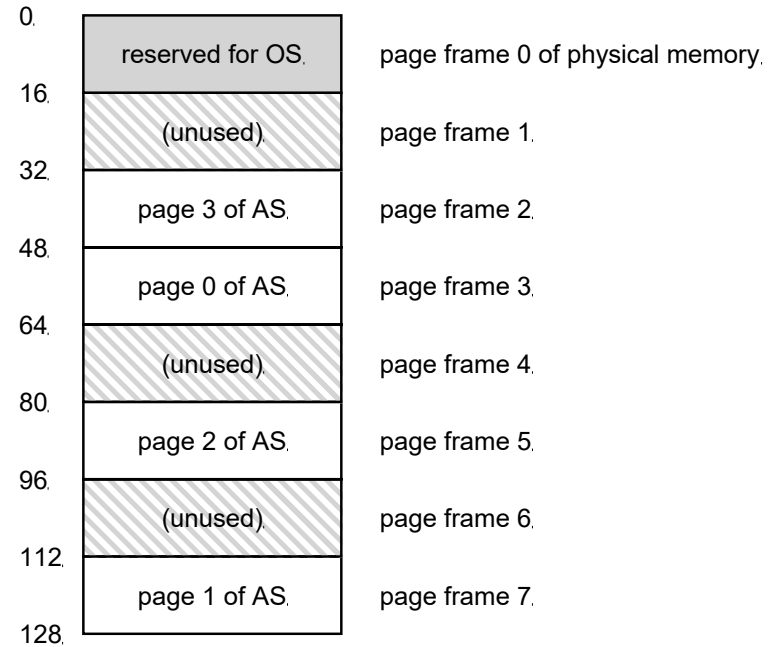


- How does the OS record where each virtual page of the address space is placed in physical memory?
 - **Per-process** data structure – **page table** – to store address translations for every virtual page of the address space
 - *How many entries in the page table of this example, and what are they?*

Virtual address space



Physical memory



The page table's content:

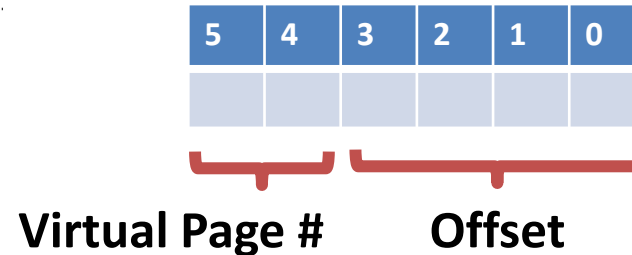
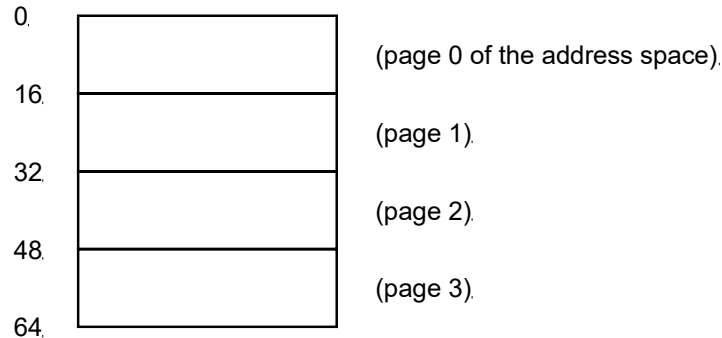
	PFN	Other bits
PTE0	3	...
PTE1	7	...
PTE2	5	...
PTE3	2	...

Address translation

- In this 64 byte address space example (16 bytes per page), how many bits does a virtual address have in this system?
- How many bits of the virtual address are needed to select a page?

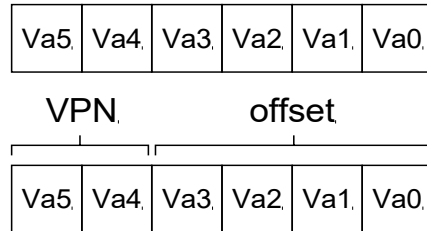
Address translation

- In this 64 byte address space example (16 bytes per page), how many bits does a virtual address have in this system?
 - $2^6 = 64$ bytes, thus **6 bit addresses**
- How many bits of the virtual address are needed to select a page?
 - 64 bytes / 16 bytes per page = 4 pages
 - $2^2 = 4$, thus **2 bits to select a page**

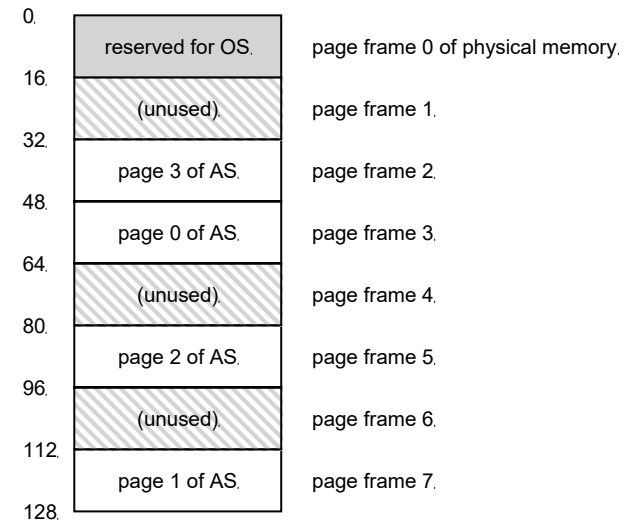
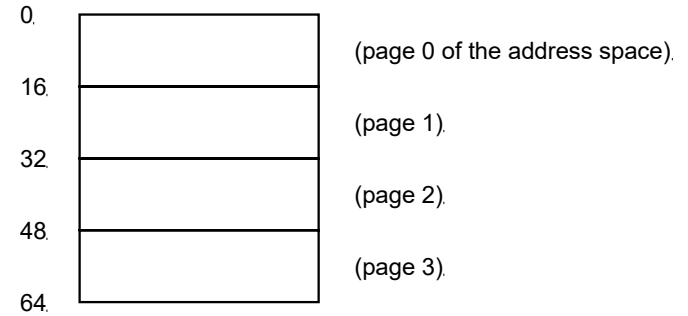
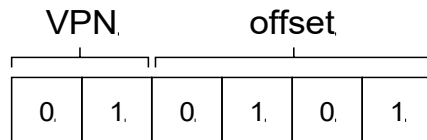


Address translation

- Split address into virtual page # (VPN) and offset
 - 64-byte address space
 - page size = 16 bytes

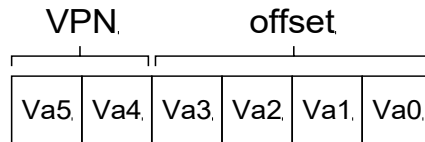
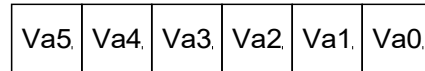


— VA 21

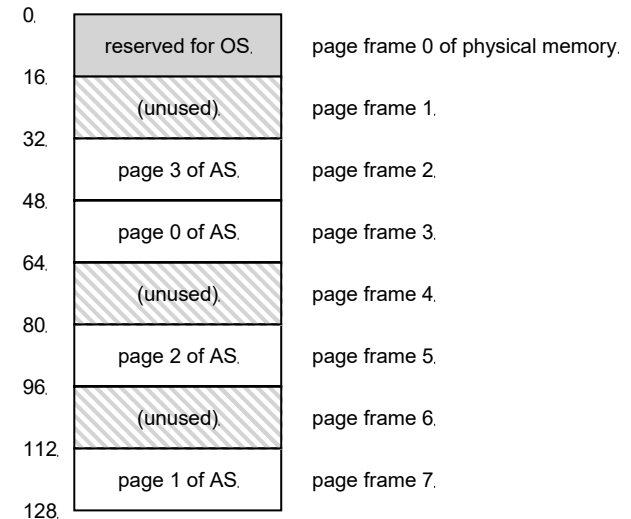
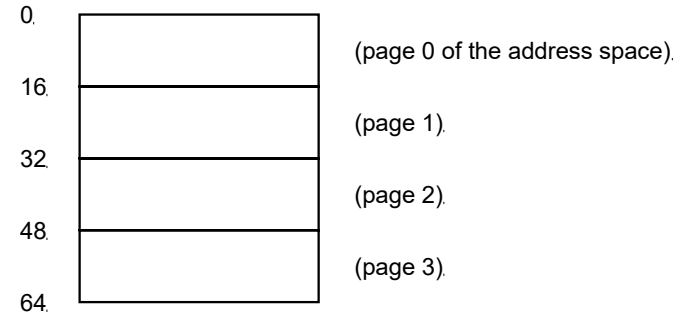
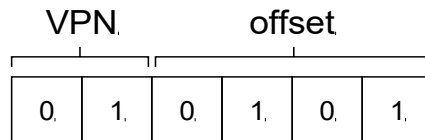


Address translation

- Split address into virtual page # (VPN) and offset
 - 64-byte address space
 - page size = 16 bytes



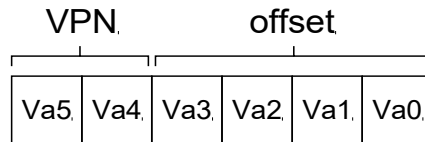
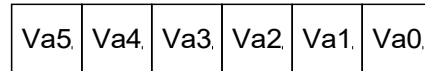
– VA 21



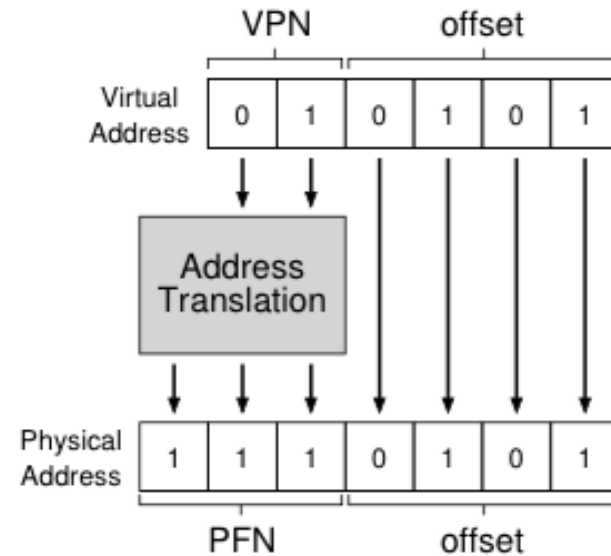
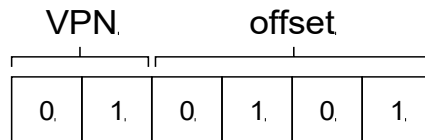
Address translation

- Split address into virtual page # (VPN) and offset

- 64-byte address space
- page size = 16 bytes



- VA 21



- **VPN indexes page table to get physical frame # (PFN)**
- **offset stays the same**

Where are page tables stored?

- Suppose a system has 32-bit VA and page size is 4KB. Answer the following three questions.
 - How many bits for VPN and offset?
 - How many pages in total?
 - Given that page table entry (PTE) size is 4 bytes, how much memory does a page table take?

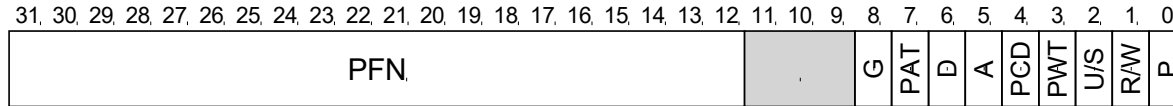
Where are page tables stored?

- Suppose a system has 32-bit VA and page size is 4KB. Answer the following three questions.
 - How many bits for VPN and offset?
 - 20-bit VPN and 12-bit offset
 - How many pages in total?
 - 2^{20}
 - Given that page table entry (PTE) size is 4 bytes, how much memory does a page table take with linear paging?
 - 2^{20} pages in the address space $\rightarrow$ 2^{20} PTEs in a PT with linear paging
 - PT size: $4 \times 2^{20} = 4 \text{ MB}$
 - Is this a big deal?
 - If there are 100 processes, 400 MB of space is needed just for storing page tables!
 - We cannot keep page tables in any special on-chip hardware. Instead, page tables are stored in “kernel” memory.

Page table organization

- Linear page table (an array, the simplest form)
 - OS indexes the array by VPN to get PFN
- Bits associated with each PTE
 - **Valid**: whether the particular translation is valid.
 - Example: unused space between stack and heap. Just mark those space as invalid. Then any access to it will generate a trap.
 - Crucial for supporting sparse address space (no need to allocate physical pages for unused space).
 - **Protection**: for R, W, X
 - **Present**: whether the page is in memory or on disk
 - **Dirty**: whether the page has been modified
 - **Reference (access)**: to track whether a page has been accessed, which is important during page replacement.

Page table organization



- Example: PTE from x86
 - Present bit (P)
 - Read/write bit (R/W)
 - User/supervisor bit (U/S)
 - Access bit (A)
 - Dirty bit (D)
 - Hardware caching related bits (PWT, PCD, PAT, and G)
 - The page frame number (PFN)

Paging is slow

- Where is the page table of the running process stored?
 - Page-table base register: physical address of the starting location of the page table
- Pseudo-code of address translation with paging

```
VPN = (VirtualAddress & VPN_MASK) >> SHIFT
PTEAddr = PageTableBaseRegister + (VPN * sizeof(PTE))
offset = VirtualAddress & OFFSET_MASK
PhysAddr = (PFN << SHIFT) | offset
```

- 32 bit address spaces (i.e., VA has 32 bits); page size: 4KB
- Values (in hexadecimal) of VPN_MASK, SHIFT, OFFSET_MASK?
- VPN_MASK = 0x FFFF F000
- OFFSET_MASK = 0x 0000 0FFF
- SHIFT = 0xC (12)

Paging is slow

Full pseudo-code:

```
// Extract the VPN from the virtual address
VPN = (VirtualAddress & VPN_MASK) >> SHIFT

// Form the address of the page-table entry (PTE)
PTEAddr = PTBR + (VPN * sizeof(PTE))

// Fetch the PTE
PTE = AccessMemory(PTEAddr)

// Check if process can access the page
if (PTE.Valid == False)
    RaiseException(SEGMENTATION_FAULT)
else
    if (CanAccess(PTE.ProtectBits) == False)
        RaiseException(PROTECTION_FAULT)
    else
        // Access is OK: form physical address and fetch it
        offset = VirtualAddress & OFFSET_MASK
        PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
        Register = AccessMemory(PhysAddr)
```

For every memory reference, paging requires to perform one extra memory reference in order to first fetch the translation from the page table.